

C# 프로그래밍 — 중간고사 실습 시험

동명대학교 게임학부
2026년 1학기

이름 :

학번 :

시험 안내

- 총 5문제, 각 20점 만점 (총 100점)
- 문제 1-4: 새 Console 프로젝트를 만들어 코드를 작성하시오.
- 문제 5: `dotnet new winforms`로 Windows Forms 프로젝트를 만드시오.
- Console 문제는 **top-level statements** 방식을 사용하시오 (Main 메서드 없음).
- Console 문제에서 클래스가 필요하면 파일 **하단**에 클래스를 정의하시오.
- Windows Forms 문제는 클래스별로 **별도 파일**에 작성하시오.
- 각 문제마다 별도의 프로젝트 폴더를 만들어 작성하시오.
- 프로젝트 이름: Midterm01, Midterm02, ..., Midterm05
- 모든 프로젝트를 학번_이름 폴더 아래에 모은 후 **zip**으로 압축하여 제출하시오.
- 예시 구조:

```
20261234_홍길동/  
  Midterm01/  
  Midterm02/  
  Midterm03/  
  Midterm04/  
  Midterm05/
```

- 시험 시간: 120분

문제 1. 변수, 형 변환, 조건문 (20점)

사용자로부터 키(cm)와 몸무게(kg)를 입력받아 BMI를 계산하는 프로그램을 작성하시오.

문제 1: BMI 계산기

요구 사항

- `Console.ReadLine()`으로 키(cm)와 몸무게(kg)를 입력받는다.
- 입력값은 `string`이므로 `double`로 형 변환한다.
- BMI 공식: $BMI = \frac{\text{몸무게(kg)}}{\text{키(m)}^2}$ (키를 cm에서 m으로 변환)
- BMI 결과에 따라 판정을 출력한다:
 - < 18.5: 저체중
 - 18.5 ~ 24.9: 정상
 - 25.0 ~ 29.9: 과체중

- ≥ 30.0 : 비만
- BMI는 소수점 첫째 자리까지 출력한다.

실행 결과 (예시)

```
=== BMI 계산기 ===
키(cm)를 입력하세요: 175
몸무게(kg)를 입력하세요: 70

키: 175.0 cm (1.75 m)
몸무게: 70.0 kg
BMI: 22.9
판정: 정상
```

Hint

- `double.Parse(Console.ReadLine())`로 문자열을 실수로 변환
- cm를 m으로: `heightM = heightCm / 100.0`
- 소수점 출력: `$"{bmi:F1}"`

문제 2. 반복문 — 숫자 맞추기 게임 (20점)

숫자 맞추기 게임을 만드시오.

문제 2: 숫자 맞추기 게임

요구 사항

- 프로그램이 1 100 사이의 랜덤 숫자를 생성한다.
- `while` 문을 사용하여 사용자가 맞출 때까지 반복한다.
- 매 시도마다 “더 큼니다” 또는 “더 작습니다” 힌트를 제공한다.
- 시도 횟수를 세어 맞췄을 때 함께 출력한다.
- 최대 시도 횟수는 10회로 제한한다. 10회 안에 못 맞추면 정답을 공개한다.

실행 결과 (예시)

```
=== 숫자 맞추기 게임 (1~100) ===

[시도 1/10] 숫자를 입력하세요: 50
힌트: 더 큼니다!

[시도 2/10] 숫자를 입력하세요: 75
힌트: 더 작습니다!

[시도 3/10] 숫자를 입력하세요: 63
정답! 3번 만에 맞추셨습니다!
```

Hint

- `Random rand = new Random(); int answer = rand.Next(1, 101);`
- 반복 조건: `while (!correct && tries < 10)`
- 입력: `int.Parse(Console.ReadLine())`

문제 3. 배열과 foreach (20점)

게임 아이템 인벤토리를 관리하는 프로그램을 작성하시오.

문제 3: 아이템 인벤토리**요구 사항**

- 아이템 이름 배열 (`string[]`)과 가격 배열 (`int[]`)을 선언하고 5개 이상의 데이터를 초기화한다.
- `foreach`를 사용하여 전체 아이템 목록을 출력한다.
- `for` 문을 사용하여 다음 통계를 계산한다:
 - 가장 비싼 아이템 (이름과 가격)
 - 가장 싼 아이템 (이름과 가격)
 - 전체 아이템 평균 가격
- 특정 가격 이상의 아이템만 필터링하여 출력한다 (기준 가격은 코드에서 지정).

실행 결과 (예시)

=== 아이템 인벤토리 ===

[전체 목록]

1. 나무 검	-	500 골드
2. 철 방패	-	1200 골드
3. 체력 포션	-	300 골드
4. 마법 지팡이	-	2500 골드
5. 가죽 갑옷	-	800 골드

[통계]

최고가: 마법 지팡이 (2500 골드)
 최저가: 체력 포션 (300 골드)
 평균가: 1060.0 골드

[1000 골드 이상 아이템]

- 철 방패 (1200 골드)
- 마법 지팡이 (2500 골드)

Hint

- 배열 초기화: `string[] names = {"나무 검", "철 방패", ...};`
- 최대값 인덱스를 추적: `int maxIdx = 0;` 후 반복문에서 비교
- 평균: 합계를 구한 후 `(double)sum / prices.Length`

문제 4. 클래스 협력 — 두 클래스 조합 (20점)

은행 계좌 시스템을 두 개의 클래스로 구현하시오.

문제 4: 은행 계좌 시스템

Account 클래스

- 필드: Owner (string), Balance (int, 초기값 0)
- 생성자: 소유자 이름을 받음
- Deposit(int amount) 메서드: 입금 (양수만 허용)
- Withdraw(int amount) 메서드: 출금 (잔액 부족 시 실패 메시지 출력, bool 반환)
- GetInfo() 메서드: 이름과 잔액 문자열 반환

Bank 클래스

- 필드: accounts (Account 배열, private), count (int, private)
- 생성자: 최대 계좌 수를 받아 배열 생성
- CreateAccount(string name) 메서드: 새 계좌를 추가
- FindAccount(string name) 메서드: 이름으로 계좌를 찾아 반환 (없으면 null)
- PrintAll() 메서드: 모든 계좌 정보 출력

top-level 코드

- Bank 객체를 생성하고, 3개 이상의 계좌를 만든다.
- 입금, 출금, 잔액 부족 시나리오를 시연한다.
- 전체 계좌 목록을 출력한다.

실행 결과 (예시)

=== 은행 시스템 ===

계좌 생성: 김민수
계좌 생성: 이서연
계좌 생성: 박지훈

김민수: 10000원 입금 → 잔액: 10000원
이서연: 5000원 입금 → 잔액: 5000원
김민수: 3000원 출금 → 잔액: 7000원
이서연: 8000원 출금 → 잔액 부족! (현재: 5000원)

=== 전체 계좌 ===

김민수 : 7000원
이서연 : 5000원
박지훈 : 0원

Hint

- FindAccount에서 for 문으로 이름 비교: if (accounts[i].Owner == name)
- 출금 실패 시 false 반환, 성공 시 true 반환

- count를 사용하여 현재 등록된 계좌 수를 추적

문제 5. Windows Forms — 바운스 볼 애니메이션 (20점)

Windows Forms 프로젝트를 만들어 여러 개의 공이 화면 안에서 튕기는 애니메이션을 구현하시오.

문제 5: 바운스 볼 애니메이션

프로젝트 생성

- `dotnet new winforms -n Midterm05`로 Windows Forms 프로젝트를 생성한다.
- `Form1.cs`를 삭제하고, 아래의 `GameForm.cs`와 `Ball.cs`를 작성한다.
- `Program.cs`에서 `Application.Run(new GameForm())`으로 실행한다.

Ball 클래스 (Ball.cs)

- 필드: `X, Y` (float — 위치), `Dx, Dy` (float — 속도/방향), `Size` (int — 공 크기), `Color` (Brush — 색상)
- 생성자: 위치, 속도, 크기, 색상을 받아 초기화
- `Update(int width, int height)` 메서드:
 - `X += Dx, Y += Dy`로 이동
 - 벽에 닿으면 방향 반전 (예: `X < 0`이면 `Dx = -Dx`)
- `Draw(Graphics g)` 메서드: `g.FillEllipse(Color, X, Y, Size, Size)`로 공을 그림

GameForm 클래스 (GameForm.cs — Form 상속)

- 필드: `Ball[]` 배열 (3개 이상), `Timer` 객체
- 생성자:
 - 윈도우 크기 설정 (800×600), 제목 설정
 - 서로 다른 색상/크기/속도의 Ball 3개 이상 생성
 - `Timer`를 16ms 간격으로 설정하고 시작
- `Timer`의 `Tick` 이벤트: 모든 Ball의 `Update()`를 호출한 후 `Invalidate()`
- `OnPaint()` 오버라이드: 배경을 검정으로 칠한 후 모든 Ball의 `Draw()`를 호출

실행 결과 설명

```
+-----+
| Midterm05 - Bounce Ball |
|                           |
|           O (Red, 40px, fast) |
|           /               |
|                           |
|                           |
|                           |
|           o (Cyan, 20px, slow) |
|           \               |
|                           |
|           O (Yellow, 30px, medium) |
|           \               |
|                           |
+-----+
```

+-----+

- 3개 이상의 공이 각자의 속도와 방향으로 이동
- 벽(상/하/좌/우)에 닿으면 반사되어 튕김
- 각 공은 서로 다른 색상과 크기

Hint

- 벽 반사 로직:

```
1 if (X < 0 || X + Size > width) Dx = -Dx;
2 if (Y < 0 || Y + Size > height) Dy = -Dy;
```

- Timer 설정:

```
1 Timer timer = new Timer();
2 timer.Interval = 16;
3 timer.Tick += (s, e) => { /* Update + Invalidate */ };
4 timer.Start();
```

- 배경 칠하기: `g.FillRectangle(Brushes.Black, 0, 0, 800, 600);`
- 공 그리기: `g.FillEllipse()` 사용

채점 기준

항목	배점	설명
프로그램 동작	8	실행 시 올바른 결과가 나오는가
코드 구조	6	클래스 설계, 메서드 분리, 적절한 자료형 사용
출력/화면 구성	3	깔끔한 출력 또는 화면 렌더링
코드 스타일	3	top-level statements, 클래스 파일 분리, 가독성
문제당 합계	20	

구분	문제	핵심 개념
기초 — Console	1	변수, 형 변환, 조건문
기초 — Console	2	while 반복문, Random
기초 — Console	3	배열, for, foreach, 통계
응용 — Console	4	클래스 2개 설계, 생성자, 메서드, 협력
응용 — Windows Forms	5	Timer, OnPaint, 클래스 분리, 애니메이션